AWS SAA（SAA-C03）　分野2 弾力性に優れたアーキテクチャの設計　#08

コンテナ

アプリをコンテナにまとめ、ECS か EKS で動かし続ける。動かす場所・権限・ネットワーク・スケーリング・デプロイの決め方

各ページの下に、根拠にした AWS の公式ドキュメント（Amazon ECS・Amazon ECR・Amazon EKS）を書いた（青い文字を押すと開く）。「」の中はドキュメントの原文どおり（日本語版は機械翻訳のため、語順が不自然なところもそのまま）。会社名や構成の例は Claude が作った例

全体像 試験ガイド タスク 2.1



図は Claude が作った模式図

タスク 2.1 の知識・スキル（このPDFに関係する部分の要約）	このPDF
コンテナのオーケストレーション（Amazon ECS、Amazon EKS など）	全体
アプリケーションをコンテナに移行する方法／コンテナを使う場面を判断する	第1・2・4章
コンポーネントのスケーリング戦略を決定する	第3・4章
ロードバランシングの概念（ALB など）	第3章

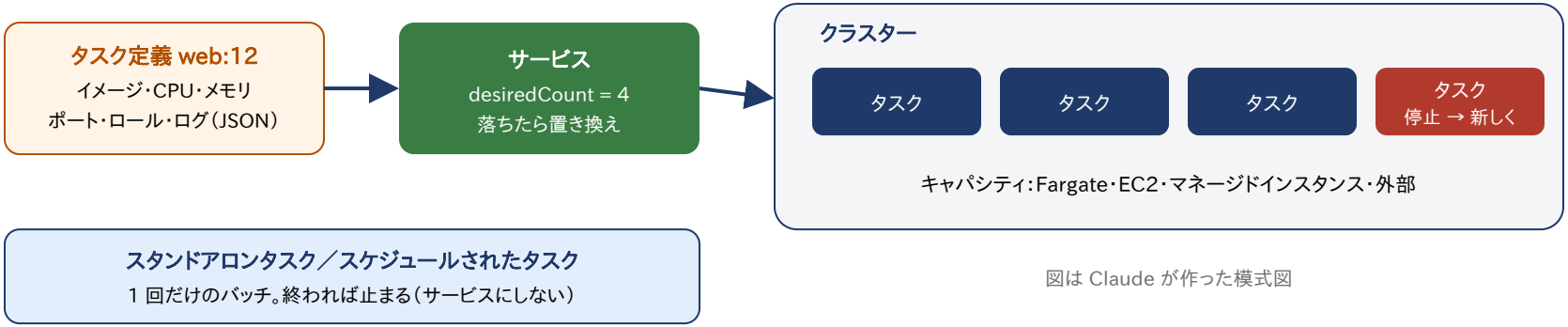
出典：AWS「コンテンツ分野 2: 弾力性に優れたアーキテクチャの設計」タスク 2.1

第1章

ECS と ECR の部品

設計図（タスク定義）から製品（タスク）を作り、係（サービス）が数を保つ。イメージは ECR に置く。

ECS の部品のつながり



図は Claude が作った模式図

サービス＝数を保つ係：決めた数のタスクを動かし続け、失敗・停止したら置き換える。ロードバランサーの後ろに置ける。Web・API など長く動くものはサービス。

REPLICA と DAEMON：レプリカ（デフォルト）は全体で数を保つ。デーモンはコンテナインスタンス（EC2）ごとに 1 つずつ（ログやモニタリングのエージェント）。

出典：Amazon ECSの タスク定義（「タスク定義はアプリケーションのブループリントです。」）／Amazon ECS サービス（「タスクの 1 つが失敗または停止した場合、Amazon ECS サービススケジューラはタスク定義の別のインスタンスを起動してそれを置き換えます。」）／Amazon ECS クラスター（「クラスターは AWS リージョン 固有です。」）

ECR:イメージの倉庫を守り、片付け、配る

機能	何をする	本番の言い回し
プライベートリポジトリ	IAM のリソースベースの権限で守る。Docker・OCI のイメージを置く	「社外に公開せずイメージを保存」
基本スキャン	AWS ネイティブの技術と CVE のデータベースで、OS の脆弱性を調べる	「プッシュ時に OS の脆弱性を」
拡張スキャン	Amazon Inspector と連携し、自動で継続的に。OS とプログラミング言語のパッケージ	「言語のライブラリも継続的に」
ライフサイクルポリシー	ルールに合う古いイメージをアーカイブ・期限切れにする	「古いイメージのストレージ料金」
レプリケーション	別のリージョン・別のアカウントへ自動でコピー	「災害対策で別リージョンにも」

だれがプルする:ECS のタスクを起動するときは、ECS(Fargate)のエージェントがタスク実行ロールで ECR からプルする(第2章)。

ひっかけ:イメージの脆弱性は GuardDuty ではなく ECR のスキャン(拡張なら Inspector)。S3 や EFS はレジストリの代わりにならない。

出典:Amazon Elastic Container Registry とは(「Amazon ECR は、IAM AWS を使用したリソースベースのアクセス許可を持つプライベートリポジトリをサポートします。」)／Amazon ECR でイメージをスキャンしてソフトウェア脆弱性がないか調べる(「拡張スキャン – Amazon ECR は Amazon Inspector と統合され、リポジトリの自動継続的なスキャンを提供します。」)／Amazon ECR でのプライベートイメージレプリケーション(「Amazon ECR は、クロスリージョンレプリケーションおよびクロスアカウントレプリケーションの両方をサポートしています。」)

第2章

キャパシティ・権限・ネットワーク

どこで動かすか、だれの権限で何を呼ぶか、タスクにどんな IP とディスクを持たせるか。

どこで動かす：起動タイプとキャパシティプロバイダー

動かす場所	サーバーの管理	向く場面
Fargate	不要	まず考える
Fargate Spot	不要・中断あり（2 分前に警告）	止まってよい処理を安く
EC2（ASG）	自分で	細かい制御・リザーブドで安く
マネージドインスタンス	AWS がパッチ・スケール	GPU・特定のインスタンス
External	自分で（オンプレ）	ECS Anywhere

起動タイプは EC2・Fargate・External の 3 つ。今はキャパシティプロバイダーで指定するのが勧められている。

キャパシティプロバイダー戦略で混ぜる（例）

FARGATE
ベース 2・重み 1

最初の 2 つは必ず FARGATE

FARGATE_SPOT
重み 3

それより上は 1:3 で配る



タスク 6 つの例

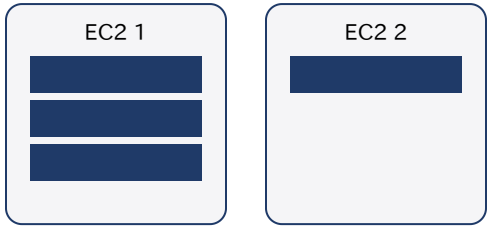
数字は Claude が作った例

ひっかけ：「Fargate Spot という起動タイプ」はない。Fargate Spot はキャパシティプロバイダー（FARGATE_SPOT）で使う。

出典：Amazon ECS の起動タイプとキャパシティプロバイダー（「起動タイプには、EC2、Fargate、External があります。」「Amazon ECS でサポートされているキャパシティプロバイダーは、Fargate および FARGATE_SPOT、セルフマネージド EC2 インスタンス用の Auto Scaling グループ、Amazon ECS マネージドインスタンスです。」）／Amazon ECS マネージドインスタンスの設計／外部インスタンスの Amazon ECS クラスター

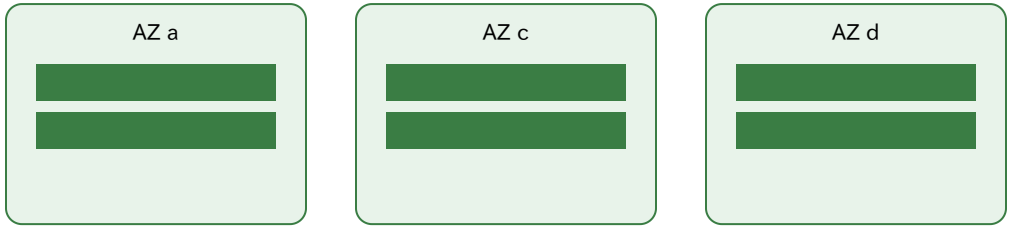
EC2 のどこに置く:タスク配置戦略

binpack(詰める)



空きを最小に → 台数が少なくて済む(コスト)

spread(散らす、AZ ごとに)

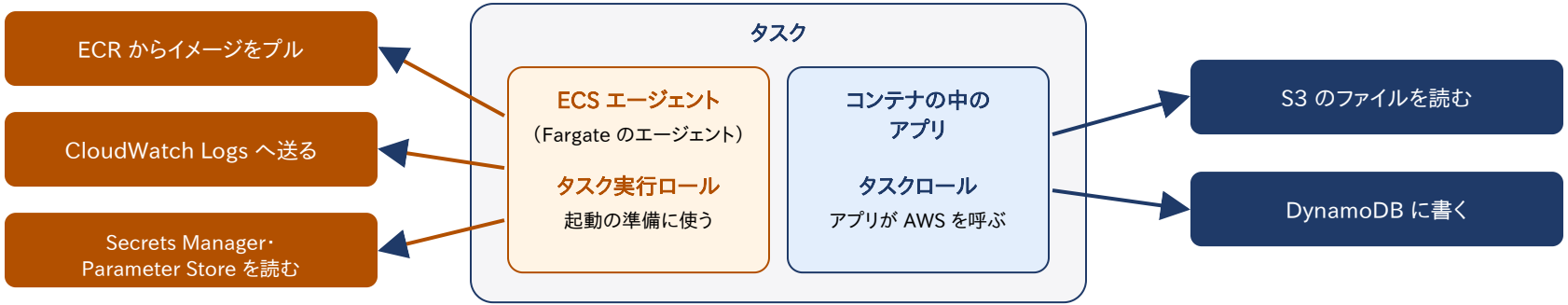


均等に → 1 つの AZ が落ちても残る(可用性)

戦略	置き方	使いどころ
binpack	未使用の CPU・メモリが最小になるよう詰める	インスタンス数を減らしてコスト削減
spread	指定した値(AZ・インスタンス)に均等に。サービスのデフォルトは AZ に spread	可用性を上げる
random	ランダム	特にこだわらない

出典:戦略を使用して Amazon ECS タスク配置を定義する(「binpack タスクはコンテナインスタンスに配置され、未使用の CPU またはメモリを最小にします。」「spreadタスクは指定された値に基づいて均等に配置されます。」)

2つのロール：タスクロールとタスク実行ロール



症状で見分ける：「イメージがプルできない」「ログが出ない」「シークレットが取れない」→ 実行ロール。「アプリが S3 で AccessDenied」→ タスクロール。

やってはいけない：EC2 のインスタンスロールにアプリの権限を付ける（全タスクが同じ権限に）・アクセスキーを環境変数に書く。DB のパスワードは Secrets Manager に置き、タスク定義の `secrets` で渡す。

出典：Amazon ECS タスクの IAM ロール（「タスクロールは、アプリケーションが Amazon S3 などの他の AWS サービスにアクセスするときに必要です。」）／Amazon ECS タスク実行IAM ロール（「タスク実行ロールは、ユーザーに代わって AWS API コールを実行するためのアクセス許可を Amazon ECS コンテナと Fargate エージェントに付与します。」）／Amazon ECS コンテナに機密データを渡す

ネットワークとストレージ

モード	しくみ
awsvpc	タスクごとに専用の ENI とプライベート IP。タスクごとにセキュリティグループ。Fargate はこれ。ターゲットタイプは ip
bridge	EC2 の中の Docker の仮想ネットワーク。動的ホストポートマッピングで ALB に登録
host	ホストの ENI のポートをそのまま使う。同じポートのタスクは 1 台に 1 つ

AWS のおすすめ: 特別な理由がなければ awsvpc。ECS Anywhere の外部インスタンスでは awsvpc は使えない。

ストレージ	特徴
Fargate のエフェメラル	最低 20 GiB、最大 200 GiB。タスクが終われば消える・共有しない
EFS ボリューム	永続・複数のタスクで共有。Fargate でも使える
EBS ボリューム	高性能なブロック。データの多い処理に

共有して残すなら EFS: アップロードされたファイルをどのタスクでも読むなら EFS。一時ファイルならエフェメラルを増やす。

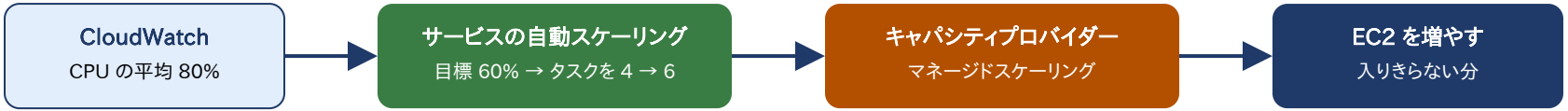
出典:EC2 の Amazon ECS タスクネットワークオプション(「Amazon ECS では、別のネットワークモードを使用する特別の必要性がある場合を除き、awsvpc ネットワークモードの使用を推奨します。」)／Amazon ECS 向けの Fargate タスクエフェメラルストレージ(「エフェメラルストレージの総量は、最大 200 GiB まで増やすことができます。」)／Amazon ECS での Amazon EFS ボリュームの使用(「これにより、タスクは、配置されているインスタンスにかかわらず、同じ永続的ストレージにアクセスできます。」)

第3章

スケーリングとデプロイ

タスクを増やし、ALB でつなぎ、止めずに入れ替える。バッチはスケジュールで起動。

タスクを増やす・インスタンスを増やす



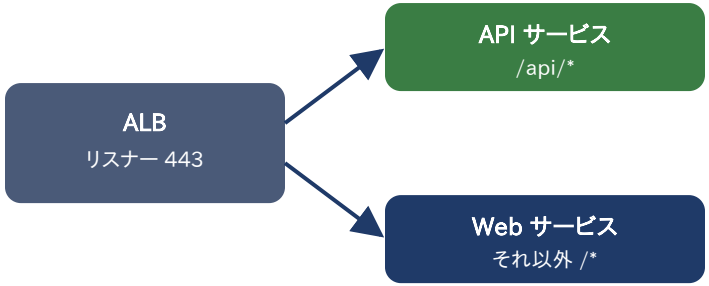
EC2 のときだけ要る。Fargate ならタスクを増やすだけでよい

数字は Claude が作った例

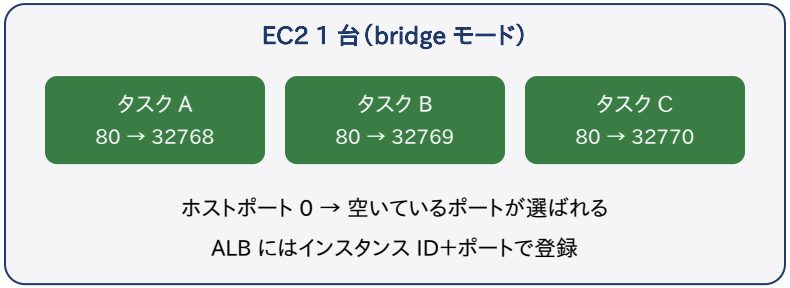
スケーリングの種類	決め方	例
ターゲット追跡	メトリクスと目標値を決めるだけ（CPU・メモリ・ALB のリクエスト数）	CPU の平均を 60% に保つ
ステップ	しきい値ごとに増減する数を決める	80% 超で +2、90% 超で +4
スケジュール	日時で数を変える	毎朝 8 時に 10 にする
予測	過去の日次・週次のパターンから前もって増やす	毎週月曜の朝の山

出典：ターゲットメトリクスを使用して Amazon ECS サービスをスケールする（「ターゲット追跡スケーリングポリシーで、メトリクスを選択してターゲット値を設定します。」）／EC2 ワークロード用の Amazon ECS キャパシティプロバイダー（「マネージドスケーリング機能を使用して、Auto Scaling グループのスケールインおよびスケールアウトアクションを Amazon ECS に管理させることも、自分自身でスケーリングアクションを管理することもできます。」）

ALB でつなぐ：パスで分ける・ポートを動的に



パスベースのルーティング（1 つの ALB で複数のサービス）



動的ホストポートマッピング（ポート番号は Claude が作った例）

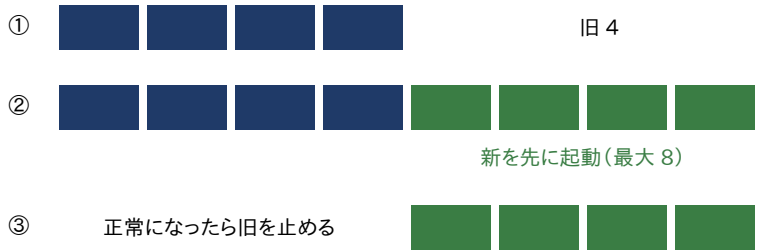
ロードバランサーの選び方：HTTP/HTTPS で URL やホストで分けるなら ALB。TCP・UDP（レイヤー 4）なら NLB。Fargate では ALB・NLB・GWLB が使える。

ターゲットタイプ：awsvpc（Fargate）のタスクは ENI に結び付くので、ターゲットグループのターゲットタイプは instance ではなく ip。

出典：Amazon ECS 用の Application Load Balancer を使用する（「Application Load Balancer では、アプリケーションレイヤー (HTTP/HTTPS) でルーティング決定を行い、パスベースのルーティングをサポートします。」「Application Load Balancer では、動的ホストポートマッピングをサポートしています。」）／Amazon ECS の AWS Fargate 用のアーキテクト（「また、このようなサービスのターゲットグループを作成する場合は、ターゲットタイプとして instance ではなく、ip を選択する必要があります。」）

止めずに入れ替える：ローリングとブルー/グリーン

ローリング更新（desiredCount 4・最小 100%・最大 200%）



正常なタスクは常に 4 以上（Claude が作った例）

minimumHealthyPercent＝正常でなければならない数の下限、maximumPercent＝動かしてよい数の上限（どちらも desiredCount に対する %）。

	ローリング	ブルー/グリーン
入れ替え方	少しずつ置き換え	新旧を両方用意して切り替え
事前の検証	しにくい	テストのトラフィックで確かめる
戻し方	もう一度デプロイ	ベイク時間中は旧版が残り、すぐ戻せる

バッチは EventBridge スケジューラ：毎日 2 時に 1 回、などはサービスにせず、スケジュールで Fargate のタスクを起動する。15 分を超えるので Lambda では無理な処理の定番。

出典：タスクを置き換えて Amazon ECS サービスをデプロイする（「maximumPercent 設定で許可されている場合、スケジューラは異常な状態のタスクを停止する前に代替タスクを起動します。」）／Amazon ECS ブルー/グリーンデプロイ（「Amazon ECS ブルー/グリーンデプロイを使用すると、本番トラフィックをルーティングする前に、新しいサービスリビジョンを検証できます。」）／Amazon EventBridge スケジューラを使用して Amazon ECS タスクをスケジュールする

第4章

Amazon EKS

Kubernetes をそのまま使いたいなら EKS。コントロールプレーンは
AWS、ノードの面倒をどこまで任せるかを選ぶ。

EKS：ノードの面倒をどこまで任せるか

コントロールプレーン（Kubernetes の頭脳）＝ AWS が管理

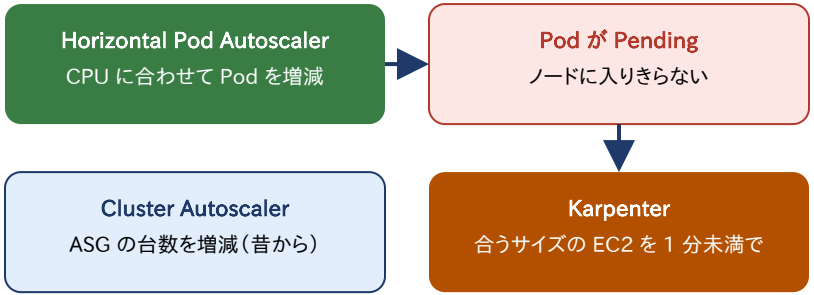
セルフマネージド	マネージド型ノードグループ	EKS Auto Mode	Fargate（プロファイル）
ノードを自分で作り 更新・パッチも自分で	作成・更新・ドレインを自動化 EC2 は自分のアカウント	インスタンス選び・スケール OS のパッチまで AWS	ノードなし。Pod ごとに VM で分離 DaemonSet 不可・プライベートのみ
手間 大			手間 小

オンプレミスで	しくみ
EKS Hybrid Nodes	自社のサーバーを、リージョンの EKS クラスターのノードにする（リージョンへの安定した接続が要る）
EKS on Outposts	データセンターに置いた AWS 管理のハードウェア（Outposts）の上で EKS
EKS Anywhere	インターネットから切り離した自己完結の環境で、Kubernetes クラスターを自分で動かす

出典：マネージドノードグループを使用してノードライフサイクルを簡素化する（「Amazon EKS マネージド型ノードグループは、Amazon EKS Kubernetes クラスターのノード (Amazon EC2 インスタンス) のプロビジョニングとライフサイクル管理を自動化します。」）／AWS Fargate を使用してコンピューティング管理を簡素化する（「デーモンセットは Fargate ではサポートされていません。」）／クラウド環境とオンプレミス環境に Amazon EKS クラスターをデプロイする（「Amazon EKS on AWS Outposts と Amazon EKS Hybrid Nodes には、オンプレミス環境から AWS リージョンへの信頼性の高い接続が必要です。」）

EKS のスケーリングと権限

2 段階のスケーリング



EKS Auto Mode は Karpenter をもとにしている

権限の渡し方	しくみ
EKS Pod Identity	Pod（サービスアカウント）ごとに IAM ロールの認証情報を渡す
IRSA	IAM ロールを Kubernetes のサービスアカウントに関連付ける
ノードのロール	全 Pod が同じ権限になる（最小権限にならない）

ECS との対応：ECS のタスクロール ≒ EKS の Pod Identity・IRSA。ECS のサービスの自動スケーリング ≒ HPA。ECS のキャパシティのスケール ≒ Karpenter。

出典：Horizontal Pod Autoscaler を使用してポッドデプロイをスケールする（「ターゲットの CPU 使用率を設定すると、Horizontal Pod Autoscaler はターゲットを満たすようにアプリケーションをスケールインまたはスケールアウトします。」）／Karpenter と Cluster Autoscaler を使用したクラスターコンピューティングのスケーリング（「EKS Auto Mode は Karpenter に基づいて構築されます。」）／EKS Pod Identity が Pod に AWS サービスへのアクセス権を付与する仕組みを学ぶ（「AWS 認証情報を作成してコンテナに配布したり、Amazon EC2 インスタンスのロールを使用したりする必要はありません。」）

ECS か EKS か

	Amazon ECS	Amazon EKS
ひとことで	AWS 独自のシンプルなオーケストレーション	Kubernetes をマネージドで
選ぶ決め手	AWS だけで動かす・覚えることを少なく	既存の Kubernetes のマニフェスト・Helm・kubectl・ノウハウ／複数の環境で同じ Kubernetes
サーバーレス	Fargate	Fargate（プロファイル）・Auto Mode
アプリの権限	タスクロール	Pod Identity・IRSA
数を増やす	サービスの自動スケーリング	HPA（Pod）・Karpenter（ノード）
オンプレミス	ECS Anywhere	Hybrid Nodes・Outposts・EKS Anywhere

本番の言い回し：「Kubernetes を使っている」「Kubernetes の知識を生かしたい」→ EKS。「運用の手間を最小に」「AWS ネイティブで」→ ECS＋Fargate。

コントロールプレーン：EC2 に自分で Kubernetes を入れると、コントロールプレーンの管理まで自分でやることになる。EKS ならコントロールプレーンは AWS。

出典：Amazon EKS とは（「Amazon Elastic Kubernetes Service (EKS) は、Kubernetes クラスターの操作の複雑性を解消する、フルマネージドの Kubernetes サービスです。」）／Amazon Elastic Container Service とは（「フルマネージドサービスである Amazon ECS には、AWS の設定と運用に関するベストプラクティスが組み込まれています。」）

選び分け：この要件ならこれ

要件	答え
常に 4 つ動かし、落ちたら置き換え	**ECS サービス**
各 EC2 にエージェントを 1 つずつ	**デーモン戦略**
言語のパッケージも継続的にスキャン	**ECR 拡張スキャン**
古いイメージを自動で片付け	**ライフサイクルポリシー**
EC2 の台数を減らしたい	**binpack**
アプリが S3 を読む	**タスクロール**
イメージがプルできない	**タスク実行ロール**
タスクごとにセキュリティグループ	**awsvpc**

要件	答え
複数タスクで永続ファイルを共有	**EFS ボリューム**
最低 2 つは通常、残りは安く	**FARGATE+FARGATE_SPOT**
1 台に同じサービスを複数+ALB	**動的ホストポートマッピング**
本番前に検証し、すぐ戻す	**ブルー/グリーン**
毎晩 40 分のバッチ	**スケジューラ+Fargate タスク**
Kubernetes をそのまま移す	**EKS**
Pod の数を CPU で増減	**HPA**
Pending の Pod のためにノード追加	**Karpenter**

出典：このPDFの各ページの資料

まとめ

ECS の部品
タスク定義＝設計図、タスク＝動いているもの、サービス＝数を保つ。イメージは ECR

動かす場所と権限
Fargate がまず候補。アプリはタスクロール、プルやログは実行ロール。awsvpc

スケーリングとデプロイ
ターゲット追跡・ALB（パス・動的ポート）・ローリング（100/200）・ブルー/グリーン

EKS
Kubernetes なら EKS。ノードはマネージド・Auto Mode・Fargate。HPA と Karpenter、Pod Identity

ひっかけ	正しくは
アプリの権限を実行ロールに	**タスクロール**
起動タイプ「Fargate Spot」	**キャパシティプロバイダー**
EKS の Fargate で DaemonSet	**使えない**
HPA でノードを増やす	**ノードは Karpenter**
Fargate の一時領域は 20 GiB まで	**200 GiB まで増やせる**

次の #09 では、ロードバランサーと Auto Scaling を学ぶ。

出典：このPDFの各ページの資料